HAZE: Adversarial Multi-Agent Scrutiny for Vulnerability Detection

Samuel Blanco
AI Safety México

Samuel Díaz
AI Safety México

Axel Morales
AI Safety México

Alex Novelo
AI Safety México

With Apart Research

Track: AI Security → Pipeline security (API, cloud)
Sub-track: Responsible AI → Hallucination mitigation & behavioral audit

---

Abstract

Single-agent LLM auditors treat every prompt as a hunt for a flaw—and often **manufacture findings** to satisfy it (Perez et al., 2022). We built **HAZE** (*Hallucination-Aware Zero-sum Examination*), a red-teaming protocol where four LLM roles debate under AI Safety via Debate (Irving et al., 2018): prosecution and defense cross-examine over a code artifact; an **isolated judge** rules only on what survived refutation. *Core claim:* single-agent auditors ask "is there a bug?"; HAZE asks "did this accusation survive refutation?" We evaluate on 10 CVEs × {vulnerable, patched} = 20 artifacts (3 repeats, majority verdict, Wilson 95% CIs, McNemar test). **Headline:** 80% recall, 30% FPR, 76.2% F1. Failures cluster into three classes—visible-sink detection (works), cross-file blind spots (misses), framing-induced false alarms (fixable)—not random debate noise. A second debate round changed zero verdicts (p=1.0). The single-agent baseline is shipped but **not yet run**; we report this explicitly.

Research conducted at the Global South AI Safety Hackathon, June 2026.

1 Introduction

Teams increasingly use LLMs to audit code, dependencies, and API pipelines—especially where dedicated AppSec capacity is scarce across the Global South. The dominant pattern—a *single agent* asked to "find the vulnerability"—suffers a systematic failure mode: RLHF-trained models confuse *"nothing is wrong here"* with *"I found nothing matching your request"*, producing false positives and hallucinated CVEs (Perez et al., 2022). There is no intrinsic self-correction: an agent thinking alone cannot demand proof from itself.

Threat model. We address *confirmation-bias hallucinations* in automated software auditing: invented findings that waste analyst time and erode trust in AI-assisted security pipelines. The same adversarial topology extends to auditing model behavior (jailbreaks, prompt injection) as a form of scalable oversight.

Conceptual foundation. HAZE builds on the AI control principle of *AI Control through Majority Voting* (Moe et al., 2025, Apart Research): **never trust a single pass** from a capable but unreliable LLM. UIAA applies this at *generation* time—query an untrusted model c times with prompt variation and aggregate outputs by majority vote via a trusted wrapper, driving backdoor probability down exponentially in c. We apply the same distrust-of-single-sample logic at *audit* time: a lone "find the vulnerability" prompt manufactures findings (Perez et al., 2022). HAZE replaces blind re-sampling with **structured adversarial scrutiny** (debate + isolated judge), while our evaluation still aggregates **majority verdicts over three independent repeats**—a direct UIAA-style redundancy layer on top of debate.

Our main contributions (new work during this hackathon):

1. **HAZE protocol & tool** — reproducible 2v2 debate + isolated judge (Fastify/OpenRouter monolith, live demo at `https://hackaton.xooktech.com/`).
2. **Ground-truth CVE benchmark** — 10 CVEs $\times$ {vulnerable, patched}, with documented copies as human ground truth.
3. **Evaluation harness & measured results** — batch runner, 120 debate transcripts, Wilson CIs, McNemar ablation, and a three-class error taxonomy.

Prior work covers multi-agent debate (Irving et al., 2018; Du et al., 2023) and LLM red teaming (Perez et al., 2022); we **adapt debate to pipeline-security auditing** and empirically measure detection on labeled CVE artifacts—not merely demonstrate a demo.

2 Related Work

AI Safety via Debate (Irving et al., 2018) frames verification as easier than generation; Khan et al. (2024) show more persuasive debaters yield more truthful judges. **Multi-agent debate** (Du et al., 2023) improves factuality over chain-of-thought. **LLM red teaming** (Perez et al., 2022; Ganguli et al., 2022) scales attack generation but typically relies on humans or classifiers for verification. **LLM-as-a-Judge** (Zheng et al., 2023) motivates our design choice to *isolate* the judge from hypothesis generation.

Gap. Red-teaming literature focuses on *eliciting* failures; debate literature targets open-domain truth. HAZE combines structured adversarial scrutiny with a labeled benchmark to ask: *can debate reduce false positives in code auditing without destroying recall?*

AI control via majority voting. Moe et al. (2025) show that querying an untrusted model multiple times and aggregating by majority vote—with a trusted consolidator—can drive backdoor success probability down exponentially in the number of samples. HAZE inherits the *never trust one pass* principle but targets the symmetric audit problem (hallucinated *findings* rather than hidden *backdoors*) and replaces output majority with adversarial cross-examination before judgment. Our three-repeat majority aggregation in evaluation explicitly follows the UIAA redundancy pattern.

3 Methods

3.1 Protocol

Given a software artifact, HAZE runs a bounded multi-turn debate: **Attacker** (Forensic Analyst + Prosecutor) cites dangerous patterns; **Defender** (Auditor + Defense Counsel) rebuts with benign context; **Judge** (isolated) reads artifact + transcript only and emits structured JSON (`verdict`, `confidence`, `riskLevel`, `keyFindings`, `reasoning`). An **Investigator** role (external CVE lookup) is designed but not implemented in this hackathon build.

Each debating role uses a **distinct LLM** via OpenRouter. Turn order alternates prosecution/defense each round (default 2 rounds, max 4). Debaters run at T=0.7; the judge at T=0.1 with forced JSON output. Prompts require citing exact lines and forbid inventing code—the burden of proof is institutional, not optional.

3.2 Implementation

Monolith in `API/`: Fastify 5 backend + vanilla JS frontend with SSE token streaming. Key modules: `roles.js` (roster), `prompts.js`, `orchestrator.js`, `openrouter.js`. Artifacts capped at 24,000 characters.

3.3 Evaluation design

Benchmark: 10 recent CVEs (2026), each in `vulnerable/` and `patched/` variants (n=20). Languages: PHP, C, Java, Python, JavaScript, NGINX config.

Run: Deployed HAZE at `https://hackaton.xooktech.com/`; roster: DeepSeek, Gemini-2.0-Flash, GPT-4o-mini,

Llama-3.1-70B (debaters), Claude Sonnet 4.5 (judge). Three repeats per artifact × two conditions (1-round and 2-round debate); majority verdict reported. `MALICIOUS` → positive prediction.

Baseline: Single-agent auditor (`evaluation/run_eval.mjs`) on identical inputs—**not executed in this run**.

What did not work (informed final design): (i) accuser-as-judge → self-confirmation; (ii) debate without citation requirements → plausible but non-executable exploits; (iii) single model for all roles → cascading errors.

4 Results

Headline (n=20 artifacts, majority over 3 repeats): **80.0% recall, 30.0% FPR, 76.2% F1** (8 TP / 3 FP / 7 TN / 2 FN).

Table 1: Detection metrics by condition (Wilson 95% CIs in brackets where measured).

Condition	Prec.	Recall	FPR	F1	Status
Single-agent baseline	—	—	—	—	Not run
HAZE debate (2 rounds)	72.7%	80.0%	30.0%	76.2%	Measured
HAZE debate (1 round)	72.7%	80.0%	30.0%	76.2%	Ablation

Debate depth (RQ3). Identical confusion matrix for 1 vs. 2 rounds; McNemar p=1.0 (n=20, 2 discordant pairs)—a second round did not change verdicts at this scale.

Table 2: Error taxonomy: three interpretable failure classes.

Class	Mechanism	n	Examples
A Visible-sink	Dangerous primitive explicit in file	8 TP	10520, 44262, 24061…
B Cross-file	Root cause outside artifact text	2 FN	42945, 32625
C Framing FP	Patched code still looks dangerous	3 FP	24061, 41089, 44815

Class C drives most false positives: asking "is this malware?" on minimal C parsers cannot distinguish patched from vulnerable. Class B explains false negatives (NGINX module bug not visible in `nginx.conf`; MCP templating reads benign). This is a *framing and scope* limitation, not unstructured debate failure.

5 Discussion and Limitations

Implications. HAZE instantiates *scalable oversight*: the judge evaluates which side survived cross-examination rather than generating the audit from scratch—the step most prone to hallucination. For pipeline security, structured debate plus full transcripts supports human review before blocking deployments. OpenRouter access means teams without local GPUs can run the protocol—relevant for Global South contexts.

Limitations. (i) No Investigator or external CVE lookup. (ii) Judge is still an LLM—structural isolation ≠ statistical independence. (iii) Single-file, 24k-char window; no runtime or multi-file analysis. (iv) **Single-agent baseline not run**—the central claim (debate reduces FPR vs. one prompt) is untested. (v) Small n=20 yields wide CIs (FPR 30% [10.8, 60.3]). (vi) Malware framing on a vulnerability benchmark inflates Class C errors. (vii) Prompts in Spanish; cross-language generalization untested. Unaddressed threats: agent collusion, prompt injection via the artifact, distributed vulnerabilities.

Dual use. HAZE could help adversaries iteratively refine malicious code until it receives `NOT_MALICIOUS`, or automate reputational harm via unreviewed `MALICIOUS` verdicts. Mitigations: human-in-the-loop before enforcement; no public unauthenticated deployments; verdicts are decision *aids*, not certificates.

Future work. Run the L0 single-agent baseline; implement Investigator with NVD/OSV; re-evaluate with `VULNERABLE/SAFE` framing (harness ready); cross-model judges; repository-scale and agent-behavior auditing;

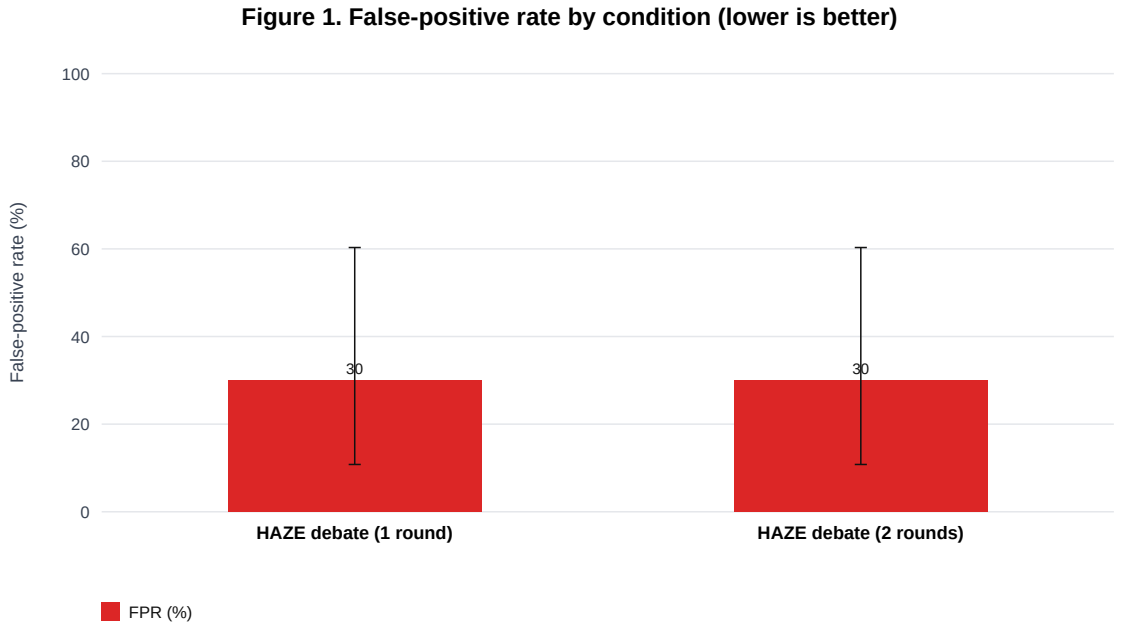

Figure 1: False-positive rate by condition (lower is better). Error bars: Wilson 95% CIs.

confidence calibration.

6 Conclusion

Single-agent auditors ask "is there a bug?" and often invent one. HAZE asks "did this accusation survive refutation?"—demonstrative scrutiny over probabilistic confidence. On a 20-artifact CVE benchmark, 2v2 debate with an isolated judge reaches 80% recall at 30% FPR; errors taxonomize into visible-sink success, cross-file blind spots, and framing-induced false alarms. Whether debate *beats* single-agent auditing on FPR remains the key empirical test; we ship the harness and report honestly that it is pending.

Code and Data

- **Repository:** `https://github.com/AXLAAF/V2-Hackathon`
- **HAZE tool:** `API/` — Fastify backend, OpenRouter orchestrator, SSE debate API, web UI
- **Live demo:** `https://hackaton.xooktech.com/`
- **Benchmark:** `vulnerabilities/` — 10 CVEs × {vulnerable, patched}
- **Evaluation:** `evaluation/` — `run_eval_web.mjs`, `run_eval.mjs`, `analyze.py`; 120 transcripts in `evaluation/results/`

Author Contributions

Samuel Blanco and Axel Morales — architecture, API implementation, and orchestration. Samuel Díaz — evaluation harness, benchmark, and metrics. Alex Novelo — writing and analysis. All authors contributed to the final report.

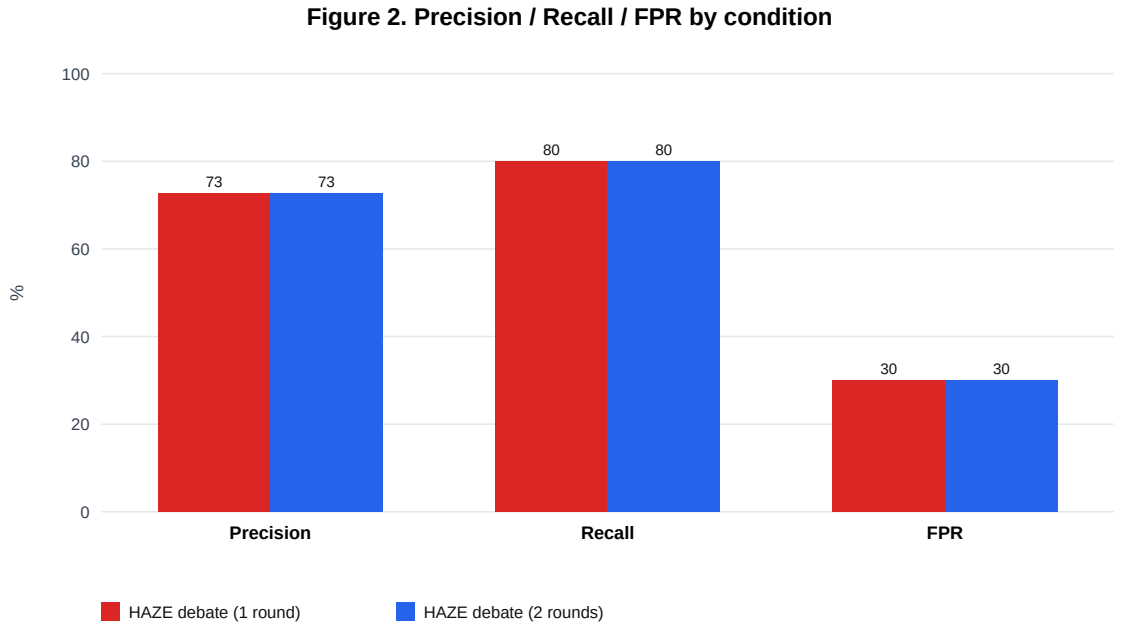

Figure 2: Precision, recall, and FPR for 1- vs. 2-round HAZE debate.

References

[1] Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., & Mordatch, I. (2023). *Improving factuality and reasoning in language models through multiagent debate*. arXiv:2305.14325.

[2] Ganguli, D., et al. (2022). *Red teaming language models to reduce harms*. arXiv:2209.07858.

[3] Irving, G., Christiano, P., & Amodei, D. (2018). *AI safety via debate*. arXiv:1805.00899.

[4] Khan, A., et al. (2024). *Debating with more persuasive LLMs leads to more truthful answers*. arXiv:2402.06782.

[5] Moe, A., Wale, W., & Sunde, J. H. (2025). *AI Control through Majority Voting*. Apart Research. `https://apartresearch.com/project/ai-control-through-majority-voting-uiaa`

[6] Perez, E., et al. (2022). *Red teaming language models with language models*. arXiv:2202.03286.

[7] Zheng, L., et al. (2023). *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. arXiv:2306.05685.

LLM Usage Statement

We used LLMs to assist with drafting, structure, and literature framing of this report. LLMs are also the system under study (the debating agents in HAZE). All technical claims, cited references, and empirical results were independently verified by the authors. The final submission was reviewed and edited by the team.